

基于本地知识库的论文搜索助手系统 — 设计开发文档

课程: 数据库系统实验 完成日期: 2026年6月

一、引言

1.1 项目背景

随着学术论文数量的爆发式增长，研究人员在阅读和检索论文时面临信息过载的问题。传统关键词搜索难以捕捉语义层面的关联，而大语言模型虽然具备强大的自然语言理解能力，但容易产生与事实不符的"幻觉"输出。本项目将检索增强生成（RAG）技术应用于本地论文知识库场景，构建了一个支持论文上传、语义检索和智能问答的助手系统，使 AI 的回答始终建立在用户上传的真实论文内容之上，兼具智能性与可信度。

1.2 设计目标

本系统的设计目标包括：构建支持 PDF 和 Word 文档上传与解析的本地知识库；实现基于向量语义检索和精排重排的论文片段检索管线；结合大语言模型生成有论文来源引用的智能回答；提供流式对话输出以提升交互体验；实现完整的用户认证和会话管理体系；采用前后端分离的微服务架构以保证系统的可维护性和可扩展性。

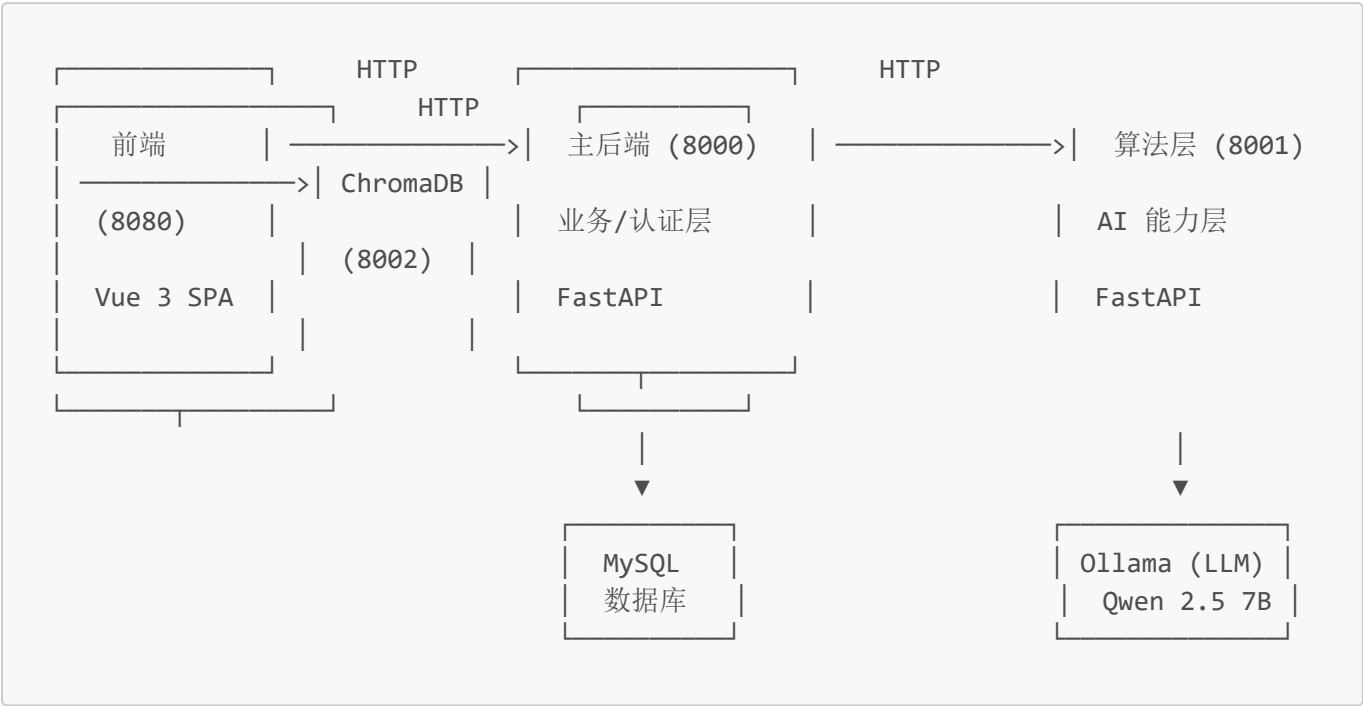
1.3 技术选型总览

层次	技术选型	选型理由
前端框架	Vue 3 + TypeScript + Vite	组合式 API 适合复杂交互，TypeScript 提升代码健壮性
UI 组件库	Element Plus	成熟的 Vue 3 组件库，表单、弹窗等组件开箱即用
状态管理	Pinia	Vue 3 官方推荐，支持 Composition API 风格
主后端框架	FastAPI	异步支持、自动 OpenAPI 文档、类型安全的请求校验
算法层框架	FastAPI	与主后端技术栈统一，便于 AI 能力的独立部署
关系数据库	MySQL（通过 PyMySQL）	成熟的 RDBMS，支持持久化存储与跨设备访问
向量数据库	ChromaDB	轻量级嵌入式向量数据库，支持 HTTP 远程访问
大语言模型	Qwen 2.5 7B（通过 Ollama）	本地部署，隐私安全，中文能力强
文本嵌入	阿里云 DashScope text-embedding-v4	高质量中文嵌入，1024 维
重排模型	阿里云 DashScope qwen3-rerank	交叉编码器精排，显著提升检索质量
文档解析	PDFPlumber + python-docx	支持 PDF 和 DOCX 双格式，表格提取能力强

二、系统架构设计

2.1 整体架构

系统采用四层微服务架构，由前端、主后端（业务层）、算法层和向量数据库四个独立服务组成。服务间通过 HTTP REST API 通信，各层职责如下：



前端只与主后端通信（通过 Vite 开发代理将 `/api/*` 请求转发到 8000 端口），主后端将涉及 AI 能力的请求转发给算法层，算法层直接与 ChromaDB、Ollama LLM 以及阿里云 DashScope API 交互。这种分层设计使认证逻辑与 AI 能力解耦，算法层可以独立升级而不影响业务层。

2.2 服务通信设计

前端与主后端之间通过 Vite 的开发代理进行通信，代理配置将 `/api/` 前缀的请求转发至 `http://127.0.0.1:8000`，并去除 `/api` 前缀。生产环境下可由 Nginx 反向代理替代。

主后端与算法层之间通过 HTTP 客户端（`httpx` 和 `requests`）直接调用，算法层暴露了 `/internal/*` 前缀的内部接口供主后端使用，包括嵌入计算、知识库清空和单篇论文删除等操作。

对话流式输出采用 Server-Sent Events (SSE) 协议。主后端在接收到算法层的流式响应后，以 `text/event-stream` 格式逐块转发给前端。前端使用原生 `fetch` API 的 `ReadableStream` 进行逐行解析。

2.3 数据流概览

以用户提出一个论文相关问题为例，完整的数据流如下：

1. 前端将用户消息和对话历史发送到 `POST /api/chat/stream`
2. 主后端接收请求，创建或获取会话，保存用户消息，然后将请求转发给算法层的 `POST /chat/stream`
3. 算法层提取用户最新消息，调用 DashScope Embedding API 获取查询向量
4. 在 ChromaDB 中执行近似最近邻 (ANN) 搜索，获取 100 个候选文本块
5. 调用 DashScope Qwen3-Rerank 进行精排，选出 Top 5 最相关片段
6. 将检索到的文本片段注入 Prompt 模板，连同完整对话历史发送给 Ollama LLM

7. LLM 以流式方式返回生成结果，算法层先输出来源标记 `[[SOURCES:...]]`，再逐块转发 LLM 输出
8. 主后端在转发流的同时累积助手回复内容，流结束后将完整回复保存到 MySQL
9. 前端实时解析 SSE 事件流，逐字展示 AI 回答并显示引用来源

三、模块设计

3.1 前端模块设计

3.1.1 路由结构

前端采用 Vue Router 的单页应用路由方案，使用 HTML5 History 模式。路由分为三个区域：公开路由（登录页 / 和注册页 `/register`，标记 `guestOnly`，已登录用户访问时自动跳转主页）、受保护路由（`/index` 及其所有子路由，标记 `requiresAuth`，未登录用户访问时跳转登录页）、以及测试路由（`/test`，不设守卫）。全局 `beforeEach` 守卫根据 Pinia 用户 Store 中的认证状态进行拦截判断。

3.1.2 组件层次

```
App.vue
├── Login.vue / Register.vue      (登录注册页)
│   └── LoginForm.vue / RegisterForm.vue
├── Index.vue                    (主布局页)
│   ├── Header.vue              (顶部导航栏 + 登出按钮)
│   ├── AsideNavBar.vue         (左侧功能菜单)
│   └── <router-view>
│       ├── Profile.vue          (用户信息页)
│       ├── CheckUserInfo.vue    (用户列表页)
│       ├── AddUser.vue          (添加用户页)
│       └── Chat.vue             (聊天主页面)
│           ├── ConversationList.vue (会话列表侧栏)
│           ├── PaperList.vue      (论文库面板)
│           ├── ChatHistory.vue    (消息展示区)
│           ├── ChatInput.vue      (输入与操作区)
│           └── ReferenceSidebar.vue (引用来源侧栏)
```

3.1.3 状态管理

系统使用两个 Pinia Store 进行全局状态管理。User Store 采用 Options API 风格，管理 `userName` 和 `token` 两个状态，初始化时从 `localStorage` 恢复登录态（解决页面刷新后登录状态丢失的问题），提供 `setAuth`、`logout` 和 `isAuthenticated` 等统一的认证操作方法。Conversation Store 采用 Composition API 风格（Setup Store），管理会话列表、当前会话 ID、当前消息列表和加载状态，提供加载、切换、新建、删除会话以及流式消息更新等方法。

3.1.4 流式通信实现

聊天组件 `Chat.vue` 使用原生 `fetch` API（而非 Axios）发起流式请求，通过 `response.body.getReader()` 获取可读流。解析逻辑逐行读取 SSE 数据，处理三类事件：`meta` 事件携带服务端自动创建的

`conversation_id`, `delta.content` 事件携带 LLM 逐步生成的文本内容, `[DONE]` 标记流结束。此外, 通过正则表达式从原始流中提取 `[[SOURCES:...]]` 来源标记。

针对 SSE 解析的一个边界问题做了特殊处理: 当 `meta` 事件恰好被 chunk 边界切割导致未被正确解析时, 组件在流结束后执行兜底逻辑——从服务端拉取最新会话列表, 确保 `conversation_id` 不会丢失。

3.2 主后端模块设计

3.2.1 认证体系

主后端采用 JWT (JSON Web Token) 方案实现无状态认证。用户注册时, 密码通过 `bcrypt` 哈希后存储。登录时, 通过 OAuth2 Password Bearer 流程验证凭据, 签发有效期 30 分钟的 JWT Token。受保护的 API 端点通过 FastAPI 的 `Depends` 依赖注入机制自动校验 Token 并获取当前用户。

前端的认证闭环由四个环节组成: 路由守卫在页面访问层面拦截未登录用户; 请求拦截器在每个 HTTP 请求中自动附加 `Authorization: Bearer <token>` 头; 响应拦截器在后端返回 401 状态码时自动清除登录态并跳转登录页; 统一的退出方法清除 Store 和 `localStorage` 中的所有认证数据。

3.2.2 请求代理模式

主后端作为 API 网关, 将涉及 AI 能力的请求透明转发给算法层。以流式对话为例, `POST /chat/stream` 端点的处理流程为: 自动创建会话 → 保存用户消息 → 发送携带 `conversation_id` 的 `meta` SSE 事件 → 使用 `httpx` 向算法层发起流式请求 → 逐块转发响应给前端 → 在转发的同时累积助手回复内容 → 流结束后使用独立的数据库会话保存完整的助手回复。

使用独立数据库会话是一个重要的设计决策: SSE 生成器的生命周期跨越了原始请求的数据库会话生命周期, 如果复用同一个会话, 会遇到连接已关闭的错误。

3.2.3 数据一致性保障

论文删除操作采用"先算法层后数据库"的事务顺序: 先将文件名转发给算法层删除 ChromaDB 中的向量数据, 确认算法层删除成功后再删除 MySQL 中的论文记录。如果算法层删除失败, MySQL 记录保持不变, 避免了向量数据残留但元数据已删除的不一致状态。会话删除则采用"先消息后会话"的顺序, 先删除关联的消息记录再删除会话本身, 以满足外键约束。

3.3 算法层模块设计

算法层是系统的 AI 能力核心, 由四个服务模块组成:

knowledge_service.py (知识管理服务) 负责文档的完整入库管线: 接收上传文件 → 调用解析器提取文本和表格 → 文本分块 → 批量向量化 → 存入 ChromaDB。同时提供知识库清空和单篇论文删除功能。

retrieval_service.py (检索服务) 编排完整的 RAG 检索与对话流程: 查询向量化 → ChromaDB 粗检索 → Qwen3-Rerank 精排 → Prompt 组装 → LLM 流式调用。

rerank_service.py (重排服务) 封装阿里云 Qwen3-Rerank 交叉编码器 API 的调用, 提供查询-文档对的相关性精排能力。当 API 不可用时优雅降级, 返回粗排结果的原始顺序。

embed.py (嵌入服务) 封装阿里云 DashScope text-embedding-v4 的调用, 支持以 10 条为单位的批量嵌入请求, 被知识入库和查询检索两个流程共用。

utils/parser.py（文档解析器） 提供 PDF 和 DOCX 双格式的文本和表格提取能力，以及多策略级联的论文标题自动提取功能。

四、数据库设计

4.1 关系数据库（MySQL）

系统使用 MySQL 存储用户、会话、消息和论文元数据四类数据，共四张表。

4.1.1 表结构

users 表

字段	类型	约束	说明
id	INT	主键, 自增	用户 ID
username	VARCHAR	唯一	用户名
email	VARCHAR	唯一, 索引	邮箱
first_name	VARCHAR		名
last_name	VARCHAR		姓
hashed_password	VARCHAR		bcrypt 哈希密码
is_active	BOOLEAN	默认 true	是否激活
is_superuser	BOOLEAN	默认 false	是否管理员

conversations 表

字段	类型	约束	说明
id	INT	主键, 自增	会话 ID
user_id	INT	外键→users.id	所属用户
title	VARCHAR(255)	默认"新对话"	会话标题
created_at	DATETIME		创建时间
updated_at	DATETIME		最后更新时间

messages 表

字段	类型	约束	说明
id	INT	主键, 自增	消息 ID
conversation_id	INT	外键→conversations.id	所属会话
role	VARCHAR(20)		角色（user/assistant）

字段	类型	约束	说明
content	MEDIUMTEXT		消息内容（最大 16MB）
created_at	DATETIME		创建时间

papers 表

字段	类型	约束	说明
id	INT	主键, 自增	论文 ID
filename	VARCHAR(512)	唯一, 索引	文件名
title	VARCHAR(1024)	默认空串	提取的论文标题
chunks_count	INT		知识块数量
uploaded_at	DATETIME		上传时间

4.1.2 表关系

用户与会话之间是一对多关系（一个用户拥有多个会话），会话与消息之间是一对多关系（一个会话包含多条消息）。论文表独立存在，不与用户表建立外键关联（当前版本论文库为全局共享）。消息的 `content` 字段使用 `MEDIUMTEXT` 类型而非标准的 `TEXT`，是因为 LLM 生成的长回复可能超过 64KB 的标准 `TEXT` 上限。

4.2 向量数据库（ChromaDB）

ChromaDB 以独立 HTTP 服务的形式运行在 8002 端口，使用持久化存储目录 `data_vector_db`。系统维护单一集合 `my_knowledge_base`，每个文本块以 `{filename}_chunk_{i}` 为唯一 ID 存储，附带 `{"source": filename}` 元数据用于标识所属论文。向量维度为 1024（对应 text-embedding-v4 模型的输出维度），相似度度量默认为余弦距离。

五、API 设计

5.1 主后端 API（端口 8000）

认证相关

方法	路径	认证	说明
POST	/token	否	用户登录，返回 JWT Token
POST	/users/	否	用户注册
GET	/users/me/	是	获取当前用户信息
GET	/users/	是	获取用户列表（分页）

对话与会话管理

方法	路径	认证	说明
POST	/chat/stream	是	流式对话（SSE），自动创建会话并持久化
POST	/chat	是	非流式对话
GET	/conversations/	是	获取当前用户的会话列表（分页，按更新时间倒序）
GET	/conversations/{id}	是	获取会话详情及所有消息
DELETE	/conversations/{id}	是	删除会话及其所有消息
POST	/conversations/{id}/messages	是	手动追加消息到指定会话

知识库与论文管理

方法	路径	认证	说明
POST	/upload	是	上传论文文件，转发算法层处理后记录到 MySQL
DELETE	/clear	是	清空知识库（所有向量和论文记录）
GET	/papers/	是	获取论文库列表
DELETE	/papers/{paper_id}	是	删除单篇论文（先删向量再删记录）
POST	/embed	是	内部嵌入接口，代理至算法层

5.2 算法层 API（端口 8001）

方法	路径	说明
POST	/chat/stream	RAG 流式对话，执行检索-增强-生成全流程
POST	/upload	接收文件，执行解析-分块-向量化-入库
POST	/internal/embed	计算文本嵌入向量
DELETE	/internal/clear	清空 ChromaDB 全部向量和本地文件
DELETE	/internal/papers/{filename}	删除指定论文的向量和本地文件
GET	/files/{filename}	静态文件服务，提供已上传论文的下载/预览

六、安全设计

6.1 认证与授权

系统采用基于 JWT 的无状态认证方案。密码存储使用 bcrypt 哈希算法（通过 passlib 库），Token 签名算法为 HS256，有效期 30 分钟。所有涉及数据操作的 API 端点均通过 FastAPI 的 `Depends(get_current_active_user)` 依赖注入进行认证校验，未认证请求返回 401 状态码。

6.2 前端安全

前端实现了完整的认证闭环：路由守卫防止未认证用户访问受保护页面；Axios 请求拦截器自动附加认证 Token；响应拦截器捕获 401 状态码并自动清除登录态；对于使用原生 `fetch` 的流式对话接口，单独增加了 401 状态码的检测和处理逻辑。

6.3 数据隔离

每个用户只能查看和操作自己的会话和消息（通过 `user_id` 外键过滤）。论文知识库在当前版本中为全局共享，所有认证用户均可访问。

七、开发环境与部署

7.1 开发环境

项目	版本/工具
操作系统	Windows
Python	3.12（通过 Conda 管理）
Node.js	18+
包管理	pip + npm
数据库	MySQL
LLM 服务	Ollama（Qwen 2.5 7B）
开发工具	Vite 开发服务器（热更新）

7.2 服务启动顺序

系统需要四个服务按顺序启动：主后端（8000）→ ChromaDB 向量库（8002）→ 算法层（8001）→ 前端开发服务器（8080）。每个服务独立运行在各自的终端进程中。

7.3 项目文件结构

```
project/
├── backend/                                # 主后端（业务层）
│   ├── main.py                            # FastAPI 应用入口、路由定义、认证逻辑
│   ├── models.py                          # SQLAlchemy ORM 模型
│   ├── schemas.py                         # Pydantic 请求/响应 Schema
│   ├── crud.py                            # 数据库 CRUD 操作
│   ├── database.py                        # 数据库连接与会话工厂
│   ├── security.py                        # 密码哈希工具
│   └── requirements.txt
├── backend_algo/                          # 算法层
│   ├── main.py                            # FastAPI 应用入口
│   ├── knowledge_service.py               # 文档入库与知识管理
│   ├── retrieval_service.py               # RAG 检索与对话编排
│   ├── rerank_service.py                  # Qwen3-Rerank 精排服务
│   ├── embed.py                           # 批量嵌入服务
│   └── schemas.py                         # 数据模型
```


├─	utils/parser.py	# 文档解析与标题提取
├─	requirements.txt	
├─	frontend/	# 前端
├─	src/	
├─	main.ts	# 应用入口
├─	App.vue	# 根组件
├─	router/index.ts	# 路由配置与守卫
├─	store/	# Pinia 状态管理
├─	user.ts	# 用户认证状态
├─	conversation.ts	# 会话管理状态
├─	request/	# HTTP 通信层
├─	http.ts	# Axios 实例（含拦截器）
├─	http_login.ts	# 登录专用 Axios 实例
├─	api.ts	# API 函数定义
├─	components/	# UI 组件
├─	pages/	# 页面组件
├─	package.json	
├─	vite.config.ts	
├─	batch_upload.py	# 批量上传工具
├─	batch_upload.bat	# Windows 启动器

八、版本迭代记录

系统经历了三个主要版本的迭代开发：

- v1.0（基础版本）** 建立了完整的 RAG 问答系统骨架，实现了用户认证、文档上传与解析、向量化存储、两阶段检索、流式对话和文献来源展示等核心功能链路。
- v1.1（体验增强版本）** 解决了聊天记录无法持久化的核心体验问题，新增了基于 MySQL 的多会话管理功能，同时对 RAG 管线进行了全链路诊断与日志优化，修复了多个前后端 Bug。
- v1.2（功能完善版本）** 新增了论文库精细化管理（列表查看与单篇删除）、论文标题自动提取、批量上传工具和前端鉴权体系全面加固（路由守卫 + HTTP 拦截器 + 状态持久化），进一步调优了 RAG 检索参数。

九、总结与展望

本系统实现了一个完整的基于 RAG 的论文搜索助手，从文档解析、向量化入库、两阶段检索到流式生成，覆盖了检索增强生成的全链路。微服务架构使各模块职责清晰，数据一致性设计保证了系统的可靠性，完整的前端鉴权闭环保障了使用安全。

后续可进一步改进的方向包括：引入混合检索策略（向量检索 + BM25 关键词检索）以提升召回率；支持多模态 PDF 内容提取（图表识别）；实现历史会话全文搜索；以及采用 Docker 容器化和负载均衡方案进行企业级部署。